DEEP LEARNING | TENSORFLOW + KERAS

Redes Neuronales

III

Optimización, regularización y evaluación
de redes neuronales

Jerson Baquero Cháves

Universidad de Cundinamarca

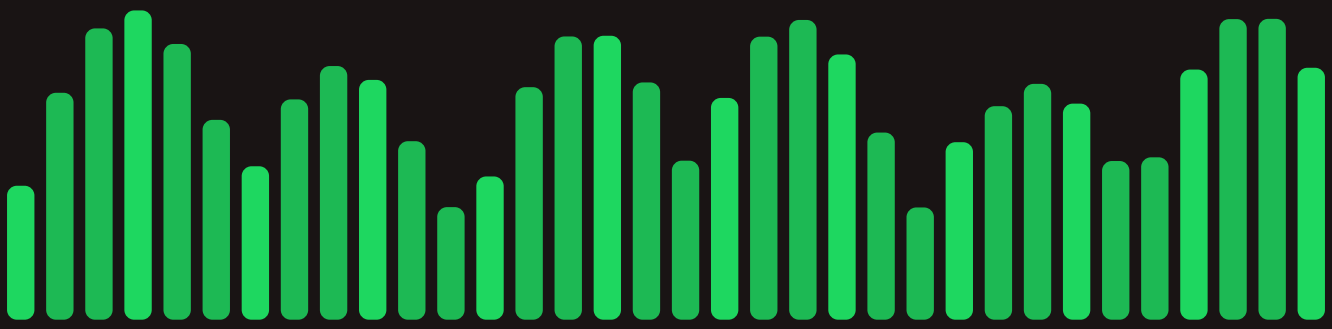
9 de octubre de 2026

GITHUB

https://github.com/Jerson09/redes-neuronales-iii

GOOGLE COLAB

https://colab.research.google.com/drive/17Qb20BlyjBpprxw69trNYZbGLccbR_DJ



1. Introducción

Entrenar una red neuronal no consiste solo en definir una arquitectura: el rendimiento final depende de cómo se distribuyen los datos, qué optimizador se usa, qué tanto se controla el sobreajuste y con qué métricas se evalúa el resultado. Este informe documenta tres prácticas desarrolladas en Google Colab con TensorFlow + Keras sobre un mismo problema de clasificación binaria: (A) comparación de optimizadores, (B) regularización con L2, Dropout y Early Stopping, y (C) evaluación final sobre un conjunto de prueba con Accuracy, Precision, Recall, F1 Score y matriz de confusión.

Todos los resultados, tablas y gráficas que aparecen aquí provienen de la ejecución del notebook entregado en el repositorio de GitHub.

2. Marco conceptual

Hiperparámetros. Son valores que el desarrollador fija antes del entrenamiento y que la red no aprende por sí sola: tasa de aprendizaje, tamaño de lote (batch), número de épocas, número de capas y neuronas, coeficiente de regularización y tasa de Dropout, entre otros. Se ajustan comparando resultados en el conjunto de desarrollo.

Conjuntos train/dev/test. El conjunto de entrenamiento (train) sirve para ajustar los pesos; el de desarrollo (dev o validación) sirve para comparar modelos e hiperparámetros y decidir cuándo detener el entrenamiento; el de prueba (test) se usa una sola vez al final para estimar el desempeño sobre datos no vistos. Usar test para tomar decisiones de diseño contamina la estimación final.

Técnicas de optimización. El descenso por gradiente estocástico (SGD) actualiza los pesos en la dirección contraria al gradiente de la pérdida. SGD con *momentum* acumula una media móvil de gradientes pasados, lo que acelera la convergencia y amortigua oscilaciones. RMSprop adapta la tasa de aprendizaje de cada parámetro dividiendo por una media móvil de los gradientes al cuadrado. Adam combina el momentum con la adaptación de RMSprop (Kingma y Ba, 2015; Ruder, 2016).

Regularización. Son técnicas que reducen el sobreajuste (overfitting). L2 penaliza la suma de los pesos al cuadrado y L1 su valor absoluto (esta última favorece pesos exactamente cero). Dropout apaga aleatoriamente una fracción de neuronas en cada paso de entrenamiento, lo que evita que la red dependa de unidades específicas (Srivastava et al., 2014). Early Stopping detiene el entrenamiento cuando la pérdida en dev deja de mejorar y puede restaurar los mejores pesos.

Bias, varianza y su tradeoff. Un sesgo (bias) alto indica que el modelo no aprende bien ni siquiera los datos de entrenamiento (subajuste); una varianza alta indica que rinde mucho mejor en train que en dev (sobreajuste). Reducir uno suele aumentar el otro: modelos más complejos bajan el sesgo pero elevan la varianza, y la regularización hace lo contrario. En este informe la varianza se aproxima con la brecha (gap) entre la exactitud de train y la de dev.

Métricas de clasificación. Accuracy es la proporción de aciertos totales. Precision es la fracción de predicciones positivas que realmente lo son (VP / (VP + FP)). Recall es la fracción de positivos reales que el modelo detecta (VP / (VP + FN)). F1 es la media armónica de Precision y Recall. La matriz de confusión muestra los cuatro tipos de resultado (VP, VN, FP y FN) y permite ver qué tipo de error se comete.

Preprocesamiento y distribución de datos. La estandarización (media 0, desviación 1) evita que las variables con escalas grandes dominen el entrenamiento. El escalador debe ajustarse únicamente con train y luego aplicarse a dev y test; ajustarlo con todos los datos filtra información del futuro (data leakage). La partición estratificada conserva la proporción de clases en cada subconjunto.

TensorFlow + Keras y PyTorch. TensorFlow es una plataforma de cómputo numérico con diferenciación automática, y Keras es su API de alto nivel para definir, compilar y entrenar modelos con pocas líneas (Abadi et al., 2015; Chollet et al., 2015). PyTorch es la alternativa más usada en investigación, con un estilo más imperativo en el que el ciclo de entrenamiento se escribe explícitamente. En esta actividad se utilizó TensorFlow + Keras.

3. Datos: distribución y preprocesamiento

Se usó el conjunto *Breast Cancer Wisconsin (Diagnostic)*, incluido en scikit-learn (Pedregosa et al., 2011; Wolberg et al., 1995): 569 muestras, 30 variables numéricas calculadas a partir de imágenes de células y una etiqueta binaria (0 = maligno, 1 = benigno). Es un problema adecuado para estudiar las métricas porque los errores de cada clase tienen consecuencias distintas.

Los datos se dividieron de forma estratificada en 60 % entrenamiento (341 muestras), 20 % desarrollo (114) y 20 % prueba (114), con semilla 42. Después se estandarizaron con `StandardScaler` ajustado solo con train.

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt, tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, ConfusionMatrixDisplay)

tf.keras.utils.set_random_seed(42)
X, y = load_breast_cancer(return_X_y=True)

# Train 60% / Dev 20% / Test 20% (estratificado)
X_tmp, X_test, y_tmp, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
X_train, X_dev, y_train, y_dev = train_test_split(X_tmp, y_tmp, test_size=0.25, stratify=y_tmp, random_state=42)

# El scaler se ajusta SOLO con train (evita data leakage)
scaler = StandardScaler().fit(X_train)
X_train, X_dev, X_test = scaler.transform(X_train), scaler.transform(X_dev), scaler.transform(X_test)
print(X_train.shape, X_dev.shape, X_test.shape)
```

Salida de la ejecución: `(341, 30) (114, 30) (114, 30)`.

Modelo y funciones auxiliares

Se definió una red densa con dos capas ocultas (64 y 32 neuronas, activación ReLU) y una neurona de salida con activación sigmoide, entrenada con entropía cruzada binaria. La función `build_model` recibe el optimizador, el coeficiente L2 y la tasa de Dropout, de modo que la misma arquitectura se reutiliza en todas las prácticas y solo cambia lo que se quiere comparar.

```
def build_model(optimizer="adam", l2=0.0, dropout=0.0):
    m = keras.Sequential([
        layers.Input(shape=(X_train.shape[1],)),
        layers.Dense(64, activation="relu", kernel_regularizer=regularizers.l2(l2)),
        layers.Dropout(dropout),
        layers.Dense(32, activation="relu", kernel_regularizer=regularizers.l2(l2)),
        layers.Dropout(dropout),
        layers.Dense(1, activation="sigmoid")])
    m.compile(optimizer=optimizer, loss="binary_crossentropy", metrics=["accuracy"])
    return m

def evaluate(model, X, y):
    pred = (model.predict(X, verbose=0) > 0.5).astype(int).ravel()
    return dict(Accuracy=accuracy_score(y, pred), Precision=precision_score(y, pred),
                Recall=recall_score(y, pred), F1=f1_score(y, pred)), pred

def plot_history(h, title):
    plt.plot(h.history["loss"], label="train"); plt.plot(h.history["val_loss"], label="dev")
    plt.title(title); plt.xlabel("Época"); plt.ylabel("Loss"); plt.legend(); plt.show()
```

Hiperparámetro	Práctica A	Práctica B
Arquitectura	30-64-32-1	30-64-32-1
Optimizador	SGD, SGD+Momentum (0.9), RMSprop, Adam	Adam (por defecto)
Tasa de aprendizaje	0.01 (SGD) / 0.001 (RMSprop, Adam)	0.001 (Adam)
Épocas	50	máx. 200 con Early Stopping
Batch size	32	32
Regularización	ninguna	L2 = 0.001; Dropout = 0.2; patience = 10

Tabla 1. Hiperparámetros usados en cada práctica.

4. Práctica A: comparación de optimizadores

Se entrenó la misma red con cuatro optimizadores durante 50 épocas y se midió el desempeño sobre el conjunto de desarrollo.

```
opts = {"SGD": keras.optimizers.SGD(0.01),
        "SGD+Momentum": keras.optimizers.SGD(0.01, momentum=0.9),
        "RMSprop": keras.optimizers.RMSprop(0.001),
        "Adam": keras.optimizers.Adam(0.001)}
rows = []
for name, opt in opts.items():
    m = build_model(optimizer=opt)
    h = m.fit(X_train, y_train, validation_data=(X_dev, y_dev), epochs=50, batch_size=32, verbose=0)
    plot_history(h, f"Optimizador: {name}")
    met, _ = evaluate(m, X_dev, y_dev); rows.append({"Optimizador": name, **met})
pd.DataFrame(rows).round(4)
```

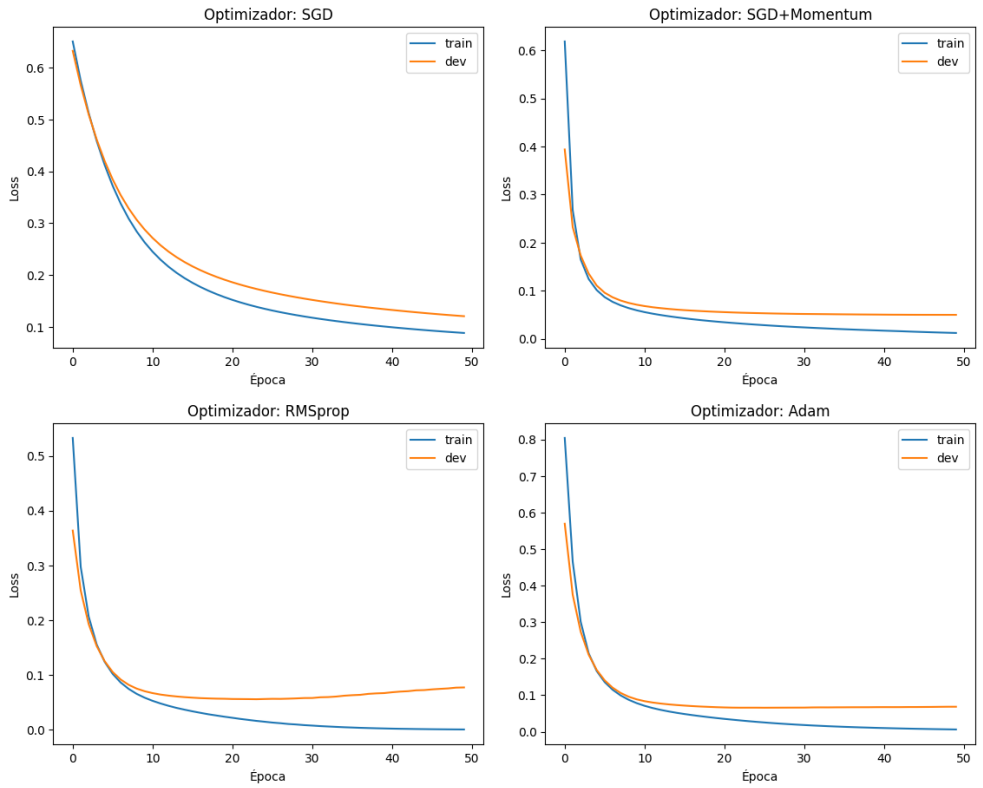


Figura 1. Curvas de pérdida (loss) en train y dev para SGD, SGD+Momentum, RMSprop y Adam.

Optimizador	Accuracy	Precision	Recall	F1
SGD	0.9561	0.9583	0.9718	0.9650
SGD+Momentum	0.9825	0.9859	0.9859	0.9859
RMSprop	0.9825	0.9859	0.9859	0.9859
Adam	0.9737	0.9857	0.9718	0.9787

Tabla 2. Métricas en el conjunto de desarrollo (114 muestras).

Análisis

- **SGD** fue el más lento: a las 50 épocas la pérdida en dev seguía bajando (cerca de 0.12), es decir, no había terminado de converger con una tasa de 0.01. También tuvo el menor desempeño (F1 = 0.9650).
- **SGD con momentum** convergió de forma mucho más rápida que SGD simple: en unas 10 épocas ya estaba cerca de su mínimo, con la menor pérdida en dev de los cuatro (cerca de 0.05).
- **RMSprop** también convergió rápido, pero la pérdida de train se acerca a cero mientras la de dev empieza a subir hacia el final (desde la época 25 aproximadamente). Es una señal temprana de sobreajuste que justifica usar Early Stopping.
- **Adam** bajó rápido y su pérdida en dev se mantuvo estable cerca de 0.065, sin la subida de RMSprop.

Las diferencias de Accuracy deben leerse con cuidado: con 114 muestras, una sola predicción equivale a 0.0088. SGD+Momentum y RMSprop acertaron 112 de 114, Adam 111 y SGD 109. Por tanto, la conclusión sólida es que los métodos con momentum o adaptativos convergen mucho más rápido que SGD simple en

este problema; la diferencia entre ellos es de una o dos muestras y con una sola ejecución no basta para declarar un ganador.

5. Práctica B: regularización (L2, Dropout y Early Stopping)

Se compararon cuatro configuraciones con el mismo optimizador (Adam): sin regularización (Baseline), L2, Dropout y L2+Dropout. Todas se entrenaron con Early Stopping sobre la pérdida de dev (paciencia de 10 épocas, restaurando los mejores pesos) y un máximo de 200 épocas. Para cada modelo se calculó la brecha entre la exactitud de train y la de dev como indicador de varianza.

Nota de buena práctica: el objeto `EarlyStopping` se crea dentro del bucle, de modo que cada modelo tiene su propio estado. Reutilizar un mismo callback entre modelos arrastra el mejor valor de pérdida del anterior y detiene prematuramente a los siguientes; esto ocurrió en una primera versión del experimento y se corrigió.

```
configs = {"Baseline": dict(), "L2": dict(l2=0.001), "Dropout": dict(dropout=0.2),
           "L2+Dropout": dict(l2=0.001, dropout=0.2)}
rows, models = [], {}
for name, cfg in configs.items():
    es = keras.callbacks.EarlyStopping(monitor="val_loss", patience=10, restore_best_weights=True)
    m = build_model(**cfg)
    h = m.fit(X_train, y_train, validation_data=(X_dev, y_dev), epochs=200,
              batch_size=32, callbacks=[es], verbose=0)
    plot_history(h, f"{name} (Early Stopping)")
    tr, _ = evaluate(m, X_train, y_train); dv, _ = evaluate(m, X_dev, y_dev)
    rows.append({"Modelo": name, "Acc train": tr["Accuracy"], "Acc dev": dv["Accuracy"],
                 "Gap (varianza)": tr["Accuracy"]-dv["Accuracy"], "Épocas": len(h.history["loss"])})
    models[name] = m
pd.DataFrame(rows).round(4)
```

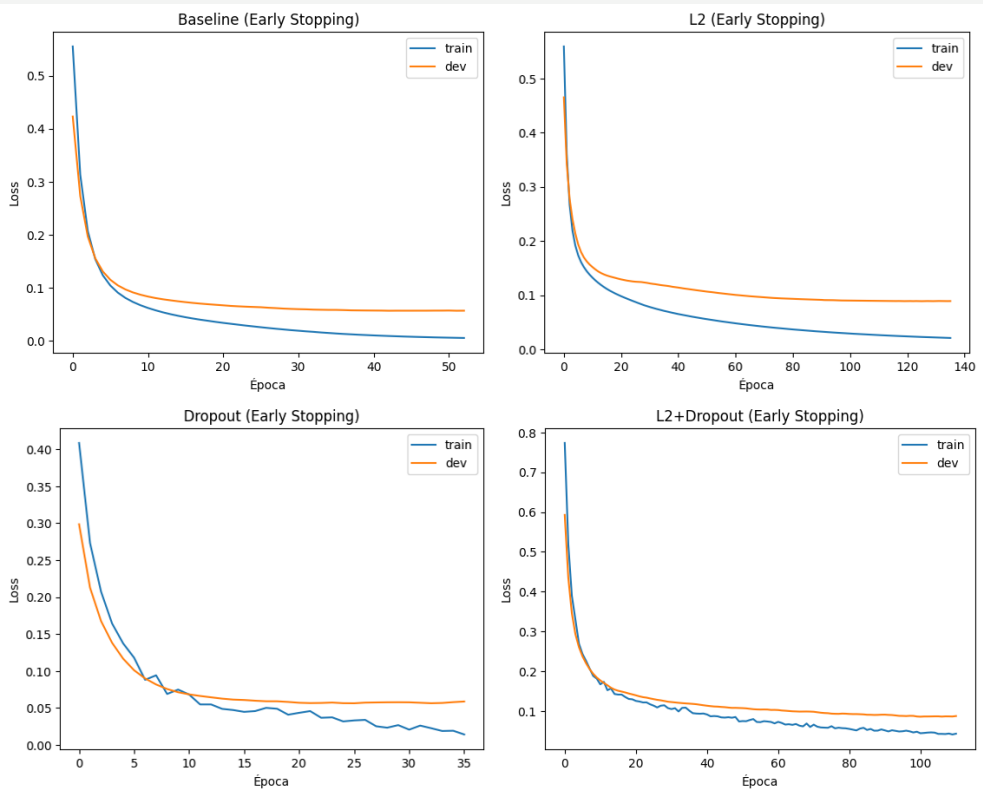


Figura 2. Curvas de pérdida con Early Stopping para Baseline, L2, Dropout y L2+Dropout.

Modelo	Acc train	Acc dev	Gap (varianza)	Épocas
Baseline	0.9971	0.9825	0.0146	53
L2 (0.001)	1.0000	0.9737	0.0263	136
Dropout (0.2)	0.9941	0.9825	0.0117	36
L2+Dropout	1.0000	0.9825	0.0175	111

Tabla 3. Exactitud en train y dev, brecha y épocas hasta la parada.

Análisis

- **Sesgo.** Todos los modelos alcanzan entre 0.994 y 1.000 de exactitud en train, por lo que el sesgo es bajo: la red tiene capacidad suficiente para el problema.
- **Varianza.** La brecha train-dev es pequeña en todos (entre 1.2 y 2.6 puntos porcentuales). Dropout obtuvo la menor (0.0117) y L2 solo la mayor (0.0263); con 114 muestras de dev, estas diferencias equivalen a una o dos predicciones.
- **Efecto de la regularización.** En este conjunto, relativamente pequeño y fácil de separar, el Baseline ya generaliza bien y ninguna técnica mejoró la exactitud en dev (Baseline, Dropout y L2+Dropout empatan en 0.9825). Dropout aportó una brecha algo menor y la curva de entrenamiento más ruidosa que es característica de apagar neuronas al azar.
- **Early Stopping.** Ningún modelo llegó al máximo de 200 épocas: se detuvieron en 53, 136, 36 y 111 épocas, es decir, ahorró entre 64 y 164 épocas según el caso, y restauró los pesos del mejor punto. Los modelos con L2 necesitaron más épocas porque la penalización hace el aprendizaje más gradual.
- **Comparabilidad de las curvas.** La pérdida de los modelos con L2 incluye el término de penalización, por lo que su valor no es directamente comparable con el del Baseline; la comparación justa se hace con las métricas de la Tabla 3.

6. Práctica C: evaluación final en el conjunto de prueba

El modelo se selecciona con el conjunto de desarrollo y solo entonces se evalúa en test. El código elige automáticamente el de mayor exactitud en dev. Como Baseline, Dropout y L2+Dropout empatan en 0.9825, la función `max` conserva el primero de la lista, que es el **Baseline**; además es el modelo más simple de los empatados, lo que es un criterio de desempate razonable.

```
best_name = max(rows, key=lambda r: r["Acc dev"])["Modelo"]
print("Modelo elegido (mejor Acc dev):", best_name)
best = models[best_name]
met, pred = evaluate(best, X_test, y_test)
print(pd.Series(met).round(4))
ConfusionMatrixDisplay(confusion_matrix(y_test, pred),
                       display_labels=["Maligno","Benigno"]).plot(cmap="Blues"); plt.show()
```



Métricas del Baseline en el conjunto de prueba (114 muestras).

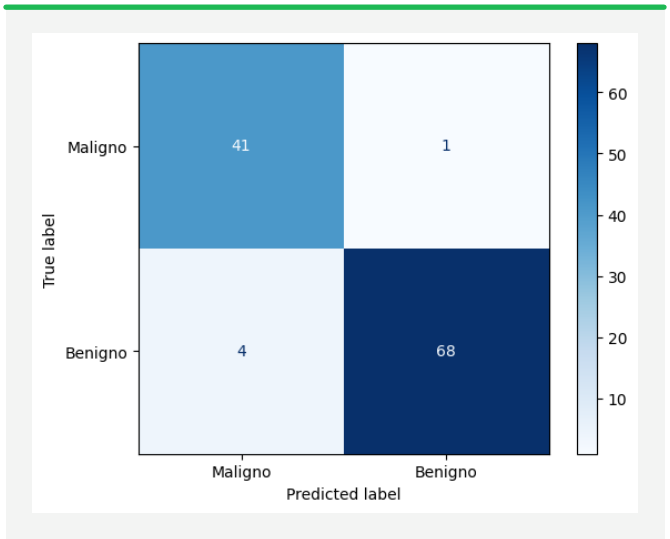


Figura 3. Matriz de confusión en test (filas: clase real; columnas: clase predicha).

Análisis

La matriz muestra 41 malignos clasificados como malignos, 68 benignos clasificados como benignos, 1 maligno clasificado como benigno y 4 benignos clasificados como malignos: 109 aciertos de 114 (Accuracy 0.9561). En scikit-learn la clase positiva es la 1, es decir, *benigno*; por eso Precision = 68/69 = 0.9855 y Recall = 68/72 = 0.9444.

El tipo de error importa más que su cantidad. Un maligno clasificado como benigno es el error costoso en un contexto clínico, porque dejaría pasar un tumor; solo ocurrió una vez (equivale a una sensibilidad de 41/42 = 0.9762 para la clase maligna). Los cuatro errores restantes son benignos marcados como malignos, que en la práctica llevarían a revisiones adicionales. Esto muestra por qué la exactitud sola no basta y se reporta también Precision, Recall y F1.

El desempeño en test (0.9561) es 2.6 puntos menor que en dev (0.9825). Es esperable: dev se usó para elegir el modelo y, con solo 114 muestras en cada conjunto, cinco errores en test frente a dos en dev bastan para explicar esa diferencia. La cifra de test es la estimación más honesta del desempeño sobre datos nuevos.

7. Conclusiones

- La distribución estratificada 60/20/20 y el escalador ajustado solo con train permitieron comparar modelos sin contaminar la evaluación final.
- Los optimizadores con momentum o adaptativos (SGD+Momentum, RMSprop, Adam) convergieron mucho más rápido que SGD simple. RMSprop mostró señales de sobreajuste hacia el final del entrenamiento.
- Early Stopping fue la técnica con efecto más claro: detuvo cada modelo entre 64 y 164 épocas antes del máximo y restauró los mejores pesos, sin necesidad de fijar el número de épocas a mano.
- L2 y Dropout no mejoraron la exactitud en dev frente al Baseline en este conjunto, que es pequeño y casi linealmente separable; su valor aquí es de control (Dropout redujo la brecha train-dev) más que de mejora. Es un resultado esperable y no un fallo de la técnica.

- El modelo final (Baseline) logró en test Accuracy 0.9561, Precision 0.9855, Recall 0.9444 y F1 0.9645, con un único falso negativo para la clase maligna.
- Limitaciones: cada experimento se ejecutó una sola vez con una semilla, y los conjuntos dev y test tienen 114 muestras, por lo que diferencias de una o dos predicciones no son concluyentes. Una validación cruzada o varias semillas darían conclusiones más firmes.

8. Repositorio y notebooks

Repositorio en GitHub con el notebook desarrollado: https://github.com/Jerson09/redes-neuronales-iii

Notebook en Google Colab: https://colab.research.google.com/drive/17Qb20BlyjBpprxw69trNYZbGLccbR_DJ

Referencias

Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., ... Zheng, X. (2015). *TensorFlow: Large-scale machine learning on heterogeneous systems*. https://www.tensorflow.org

Chollet, F., et al. (2015). *Keras*. https://keras.io

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825-2830.

Ruder, S. (2016). An overview of gradient descent optimization algorithms. *arXiv*:1609.04747.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research, 15*(56), 1929-1958.

Wolberg, W. H., Street, W. N., & Mangasarian, O. L. (1995). *Breast Cancer Wisconsin (Diagnostic)* [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C5DW2B